

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.2

Loops & Conditionals

Day 2 — teaching the computer to decide and to repeat

90 min

Concepts + live code

30 min

Practice notebook

4

Control-flow tools

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 1

1 **input() returns what type?**

→ str — always. Cast it.

2 **int(9.99) gives?**

→ 9 — it truncates, never rounds.

3 **Why is `0.1 + 0.2 == 0.3` False?**

→ Binary cannot store 0.1 exactly. Use `math.isclose()`.

4 **`2 ** 3 ** 2` evaluates to?**

→ 512 — `**` is right-to-left.

5 **When do I use `is` instead of `==`?**

→ Only with None, True and False.

Today's material sits directly on top of these. If any felt shaky, revisit the Day 1 notebook tonight.

How today's two hours are structured

1

Conditionals

if · elif · else · indentation · ternary

25 min

2

for loops

range() · enumerate · zip · nested loops

25 min

3

while loops

Conditions, infinite loops, input validation

20 min

4

break · continue · pass

Changing a loop's path mid-flight

15 min

5

Mini build

Number guessing game

5 min

6

Practice notebook

You code, I walk around

30 min

Same three notebooks as Day 1: teaching notebook · practice questions · practice solutions

By the end of today you will be able to

1

Branch

Write if / elif / else chains that pick exactly one path

2

Repeat

Loop over any sequence with for, and over a condition with while

3

Choose

Know instantly whether a task needs a for loop or a while loop

4

Control

Exit early with break, skip an item with continue, stub with pass

The habit to build today

Trace the loop by hand before you run it. Write the variable values for the first two passes on paper. Students who can hand-trace a loop debug in minutes; students who cannot debug by randomly changing things.

1

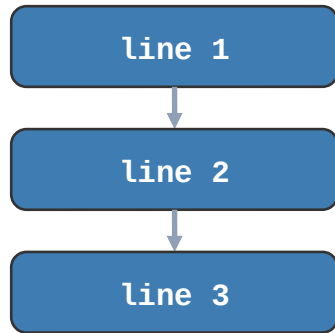
Conditionals

Teaching the program to choose · 25 minutes

- Indentation
- if / else
- elif chains
- Truthiness
- Ternary

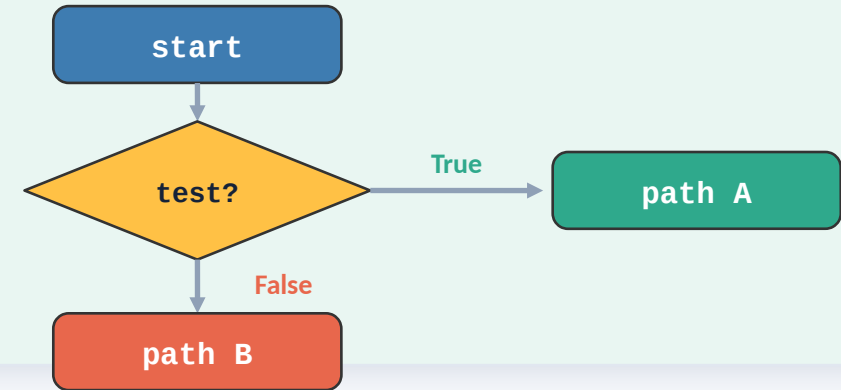
Control flow, and why indentation is not optional

Until now: straight-line code



Every line runs, once, top to bottom.
No choices, no repeats.

From today: the path can fork and repeat



! Python uses INDENTATION where other languages use { }

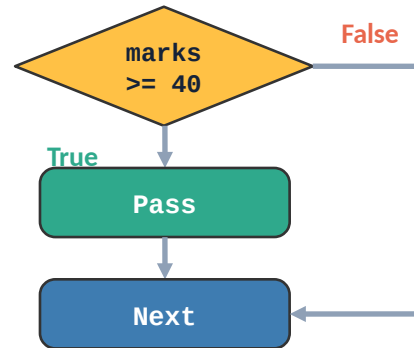
```
if marks >= 40:
    print("Pass")           # 4 spaces = inside the if
    print("Well done")     # also inside
print("Done")              # NOT indented = always runs
```

- The colon : opens a block. The indent defines it.
- 4 spaces is the standard. Be consistent — never mix tabs and spaces.
- Wrong indent = IndentationError, or worse, silently wrong logic.

if and if / else

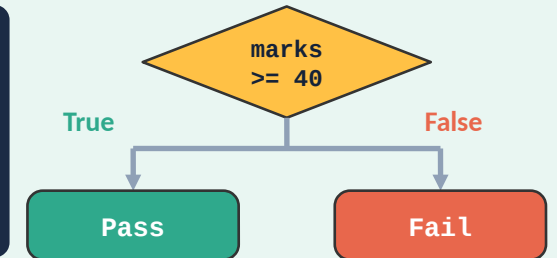
if — do it, or skip it

```
if marks >= 40:  
    print("Pass")  
  
print("Next")
```



if / else — always exactly one of the two

```
if marks >= 40:  
    print("Pass")  
else:  
    print("Fail")
```



The condition is just a bool — anything truthy works

```
if name:           # True when name is not empty  
if items:          # True when the list has items  
if count != 0:     # explicit is clearer for numbers
```

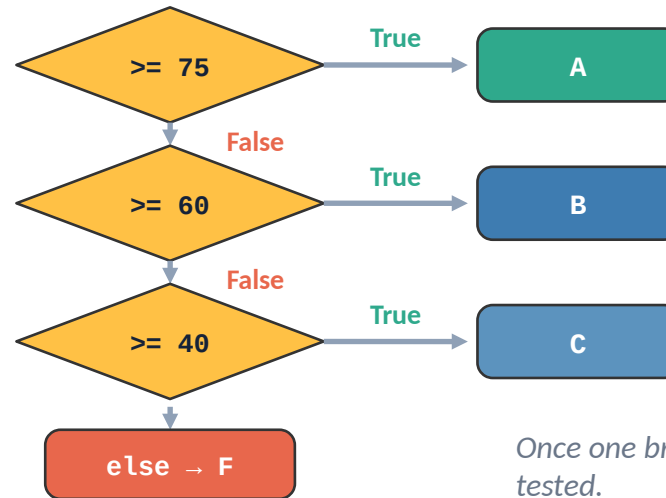
Yesterday's falsy list applies here directly: 0, 0.0, "", [], {}, (), None and False all send you down the else branch. Everything else takes the if branch — including "0" and "False", which are non-empty strings.

elif — picking exactly one from many

Grade bands

```
if percentage >= 75:  
    grade = "A"  
elif percentage >= 60:  
    grade = "B"  
elif percentage >= 40:  
    grade = "C"  
else:  
    grade = "F"  
  
# Python checks TOP TO BOTTOM and stops  
# at the FIRST True. Order matters.
```

How Python walks the chain



Once one branch fires, the rest are never tested.

⚠ Order matters: if you put `>= 40` first, a student with 90 gets a “C” — because `90 >= 40` is True and Python stops there. Always order from the most restrictive condition to the least.

Four conditional mistakes you will make this week

1 Separate ifs instead of elif

```
if m >= 75: g = "A"  
if m >= 60: g = "B"    # runs too!  
# m=90 ends up as "B"
```

Fix: use elif so only one fires.

2 Comparing to True

```
if is_valid == True:    # noisy  
if is_valid:            # correct  
  
if not is_valid:        # for False  
# never write == False
```

3 Repeating the variable

```
if m > 40 and m < 100:    # works  
if 40 < m < 100:         # better  
  
if x == 1 or x == 2:     # works  
if x in (1, 2):         # better
```

4 Empty block

```
if condition:  
    # TODO later  
    -> IndentationError
```

Fix: use `pass` as a placeholder.

Two shorthands worth knowing

Conditional expression (“ternary”)

```
# the long way
if n % 2 == 0: label = "even"
else:         label = "odd"
```

```
# the same thing in one line
label = "even" if n % 2 == 0 else "odd"
```

Reads as: value_if_true if condition else value_if_false

match / case (Python 3.10+)

```
match command:
    case "start":
        run()
    case "stop":
        halt()
    case _:          # _ is the default
        print("unknown")
```

Useful for many exact values. An elif chain does the same job.

Use the ternary only when it stays readable

```
# fine
status = "pass" if m >= 40 else "fail"
```

```
# unreadable - use if/elif instead
g = "A" if m>=75 else "B" if m>=60 else "C"
```



for loops

Repeating once per item · 25 minutes

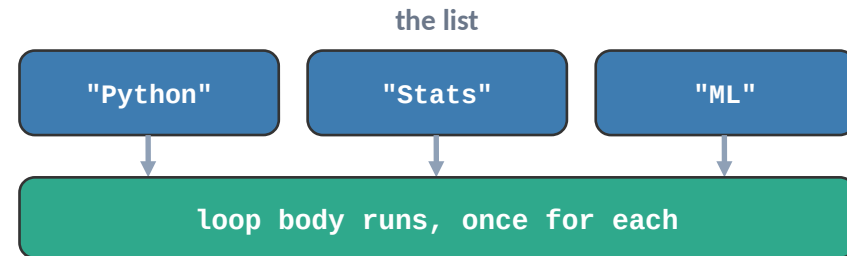
- Iterables
- `range()`
- `enumerate` & `zip`
- Accumulators
- Nested loops

The for loop — one pass per item

for <variable> in <iterable> :

```
for subject in ["Python", "Stats", "ML"]:  
    print(subject)
```

runs 3 times, once per item



What counts as an iterable?

```
for c in "PYTHON":      # each character  
for n in [1, 2, 3]:     # each list item  
for k in {"a": 1}:      # each dict KEY  
for i in range(5):      # 0,1,2,3,4  
for line in open(f):    # each file line
```

Anything you can step through, one at a time.

⚠ The loop variable is a copy

```
nums = [1, 2, 3]  
for n in nums:  
    n = n * 10      # changes n only  
print(nums)        # [1, 2, 3] - unchanged!
```

To build a changed list, collect into a new one.

range() — generating numbers to loop over

Call	Produces	Reads as
<code>range(5)</code>	0 1 2 3 4	five values, starting at 0
<code>range(2, 7)</code>	2 3 4 5 6	from 2 up to but NOT including 7
<code>range(0, 10, 2)</code>	0 2 4 6 8	step of 2
<code>range(5, 0, -1)</code>	5 4 3 2 1	counting down with a negative step

⚠ The stop value is **EXCLUDED** — the #1 off-by-one bug



`range(5)` gives these five

5 is **NOT** included

To loop 1..5 inclusive, write `range(1, 6)`. To loop over positions in a list, `range(len(items))` is exactly right.

Typical uses

```
for i in range(3):
    print("hello")

for i in range(1, 11):
    print(5, "x", i, "=", 5*i)
```

enumerate and zip — the two you will use constantly

enumerate — item AND its position

```
# clumsy
for i in range(len(names)):
    print(i, names[i])
```

```
# idiomatic
for i, name in enumerate(names):
    print(i, name)
```

```
enumerate(names, start=1) # count from 1
```

zip — walk two sequences together

```
names = ["Ravi", "Sara", "Amit"]
scores = [88, 91, 76]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

zip stops at the SHORTER sequence — extra items are silently dropped.

Looping over a dictionary

```
marks = {"Python": 88, "Stats": 71}

for subject in marks: # keys only
    pass
for subject, m in marks.items(): # key AND value
    pass
```

Looping a dict directly gives you the keys, not the values — a surprise for most people the first time. Use `.items()` when you need both, `.values()` when you only need the values. This is exactly the pattern you will use on DataFrame columns in Module 1.14.

The accumulator pattern — the backbone of data work

The three-step shape

```
total = 0                # 1. INITIALISE

for m in marks:          # 2. LOOP
    total = total + m     # 3. UPDATE
    # or: total += m

print(total)             # use it after
```

Hand-trace it: marks = [10, 20, 30]

Pass	m	total before	total after
start	—	—	0
1	10	0	10
2	20	10	30
3	30	30	60

Sum

```
total = 0
total += m
```

Count

```
count = 0
if m >= 40: count += 1
```

Maximum

```
best = marks[0]
if m > best: best = m
```

Collect

```
passed = []
passed.append(m)
```

Nested loops — a loop inside a loop

The inner loop runs fully, every outer pass

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, j)  
  
# 3 outer x 3 inner = 9 lines of output
```

```
1 1  1 2  1 3  
2 1  2 2  2 3    <- j resets every time i advances  
3 1  3 2  3 3
```

Practical: multiplication table

```
for i in range(1, 6):  
    for j in range(1, 6):  
        print(f"{i*j:4}", end="")  
    print()      # newline after each ROW
```

The bare print() sits in the OUTER loop — that is what ends each row. Move it one indent deeper and the whole table collapses into a single column. Indentation is the logic here, not decoration.

Cost grows by multiplication, not addition

Two nested loops over 1,000 items each is 1,000,000 iterations. Three levels is a billion. This is precisely why NumPy's vectorised operations exist — you will replace these loops in Topic 1.12 and watch the same job run a hundred times faster.

3

while loops

Repeating until something changes · 20 minutes

- Anatomy
- Infinite loops
- for vs while
- Input validation

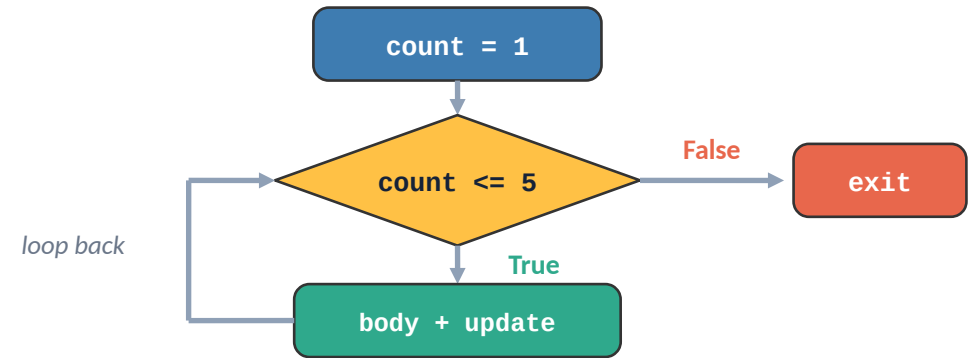
The while loop — repeat while a condition holds

Three parts you must always write

```
count = 1           # 1. INITIALISE  
  
while count <= 5:   # 2. CONDITION  
    print(count)  
    count += 1      # 3. UPDATE <- critical
```

Forget step 3 and the condition never becomes False. The loop runs forever.

The cycle



The infinite loop — and how to stop one

```
count = 1  
while count <= 5:  
    print(count)      # count never changes -> forever
```

In Colab: press the stop button on the running cell, or Runtime → Interrupt execution. Your notebook is not broken. Every programmer writes an infinite loop in their first week — the skill is recognising it within five seconds and stopping it.

for or while? Ask one question

“Do I know, before I start, how many times this should run?”

Yes → for loop

No → while loop

for — a known, finite collection

- Every row in a dataset
- Every character in a string
- Exactly 10 times (`range(10)`)
- Every file in a folder

while — until something changes

- Until the user types valid input
- Until a game ends
- Until a value converges (model training!)
- Until a server responds

In practice you will write for loops perhaps nine times out of ten. Reach for while only when the stopping point genuinely depends on something you cannot count in advance.

The classic while loop: refusing to accept bad input

Keep asking until the answer is valid

```
marks = -1

while marks < 0 or marks > 100:
    raw = input("Enter marks (0-100): ")

    if raw.isdigit():
        marks = int(raw)
    else:
        print("Digits only, please.")

print(f"Recorded: {marks}")
```

Why this shape works

- marks starts at -1 so the condition is True on the first check — the loop always runs at least once
- The condition describes what is WRONG, so the loop continues while the input is bad
- isdigit() screens the text before int() sees it, so no ValueError
- It exits only when a genuinely valid number has been stored

Yesterday's lesson, applied: input() hands you a string. Screen it, then cast it. This loop is the standard way to do both safely, and you will reuse it in almost every interactive program you write.

4

break · continue · pass

Changing a loop's path mid-flight · 15 minutes

- break: leave early
- continue: skip one
- pass: do nothing
- for / else

break and continue

break — leave the loop entirely, right now

```
for n in [1, 2, 3, 4, 5]:  
    if n == 3:  
        break  
    print(n)           # prints 1, 2 then stops
```



printed break never reached

Use it when you have found what you were searching for — there is no point checking the rest.

continue — skip this item, carry on

```
for n in [1, 2, 3, 4, 5]:  
    if n == 3:  
        continue  
    print(n)           # prints 1, 2, 4, 5
```



printed skipped printed

Use it to filter out items you do not care about, keeping the loop body clean.

In nested loops, **break** and **continue** affect only the INNERMOST loop they sit in. Breaking out of both requires a flag variable or wrapping the work in a function and using **return** — which we meet in Topic 1.3.

pass, and the for / else nobody expects

pass — a block that does nothing

```
if score > 90:
    # TODO: add bonus logic

-> IndentationError: expected an indented block
```

```
if score > 90:
    pass          # valid placeholder, does nothing
```

for / else — else runs if no break happened

```
for n in numbers:
    if n < 0:
        print("Found a negative")
        break
else:
    print("All values were positive")

# else fires ONLY if the loop finished
# without ever hitting break
```

Read for / else as “for ... if no break”

The keyword is badly named — the else has nothing to do with the if/else you learned an hour ago. It is a neat way to express “I searched the whole thing and found nothing” without a found = False flag variable. You will meet it in other people's code, so recognise it; whether you write it yourself is a matter of taste.

Debug corner — the eight loop bugs you will actually hit

You wrote	What happens	Fix
<code>range(1, 5)</code> for <code>1..5</code>	stops at 4	<code>range(1, 6)</code>
<code>while</code> with no update	runs forever	change the variable inside the loop
<code>if / if</code> instead of <code>if / elif</code>	several branches fire	<code>elif</code>
<code>for n in nums: n = n*2</code>	list unchanged	append to a new list
<code>print()</code> indented too deep	prints every pass, not per row	move it out one level
empty <code>if</code> block	<code>IndentationError</code>	<code>pass</code>
<code>continue</code> expecting a stop	loop carries on	<code>break</code>
removing items while looping	items get skipped	loop over a copy: <code>nums[:]</code>

All eight appear in today's practice notebook. If you can predict each one without running the code, you have the session.

Putting it together — number guessing game

```
import random
secret = random.randint(1, 100)
attempts = 0
MAX_TRIES = 7

while attempts < MAX_TRIES:           # unknown count
    raw = input("Your guess (1-100): ")
    if not raw.isdigit():              # validation
        print("Digits only.")
        continue                     # retry
    guess = int(raw)
    attempts += 1                     # accumulator
    if guess == secret:
        print(f"Correct in {attempts} tries!")
        break                        # found it
    elif guess < secret:               # three-way
        print("Too low")
    else:
        print("Too high")
else:
    print(f"Out of tries. It was {secret}.")
```

**while**

Unknown number of rounds

**if / elif / else**

Three-way comparison

**continue**

Reject bad input, ask again

**break**

Stop the moment they win

**while / else**

Fires only if they never broke out

**accumulator**

attempts += 1 counts the rounds

Every construct from today, in twenty-one lines.

Practice session — 30 minutes



Predict the output

Trace the loop on paper BEFORE running it

8 questions



Spot & fix the bug

Off-by-one, infinite loop, if vs elif

4 questions



Write the code

FizzBuzz, star pattern, grade counter, search

5 tasks

Your notebook

[Day2_Practice_Questions.ipynb](#)

- Open it in Colab
- Trace before you run
- Raise a hand — I am walking around
- Solutions shared after class

If a cell runs forever, press stop. That is not a mistake to be embarrassed about — it is today's material working exactly as described.

Recap

- 1 A colon opens a block; the indent defines it.
- 2 if / elif / else: only ONE branch ever runs.
- 3 elif order matters — most restrictive first.
- 4 Separate ifs are not an elif chain.
- 5 for = known count. while = until it changes.
- 6 range(stop) excludes stop. 1..5 is range(1, 6).
- 7 enumerate for index + item; zip for two lists.
- 8 Accumulator: initialise, loop, update.
- 9 Nested loops multiply cost — $1000 \times 1000 = 1\text{M}$.
- 10 A while with no update runs forever. Press stop.
- 11 break leaves; continue skips; pass does nothing.
- 12 break and continue hit the innermost loop only.

Homework

Extend the guessing game with a hot/cold hint · Print a pyramid of stars of height n · Sum every third number from 1 to 100

Next class — Topic 1.3

Functions & scope: def, arguments, default and keyword args, *args / **kwargs, return values, local vs global